

Planning from Pixels in Atari with Learned Symbolic Representations

Andrea Dittadi,* Frederik K. Drachmann,* Thomas Bolander

Technical University of Denmark, Copenhagen, Denmark
adit@dtu.dk, fdrachmann@hotmail.dk, tobo@dtu.dk

Abstract

Width-based planning methods have been shown to yield state-of-the-art performance in the Atari 2600 domain using pixel input. One successful approach, RolloutIW, represents states with the B-PROST boolean feature set. An augmented version of RolloutIW, π -IW, shows that learned features can be competitive with handcrafted ones for width-based search. In this paper, we leverage variational autoencoders (VAEs) to learn features directly from pixels in a principled manner, and without supervision. The inference model of the trained VAEs extracts boolean features from pixels, and RolloutIW plans with these features. The resulting combination outperforms the original RolloutIW and human professional play on Atari 2600 and drastically reduces the size of the feature set.

Introduction

Width-based search algorithms have in the last few years become among the state-of-the-art approaches to automated planning, e.g. the original Iterated Width (IW) algorithm (?). As in propositional STRIPS planning, states are represented by a set of propositional literals, also called boolean *features*. The state space is searched with breadth-first search (BFS), but the state space explosion problem is handled by pruning states based on their *novelty*. First, a parameter k is chosen, called the *width parameter* of the search. Searching with width parameter k essentially means that we only consider k literals/features at a time. A state s generated during the search is called *novel* if there exists a set of k literals/features not made true in any earlier generated state. Unless a state is novel, it is immediately pruned from the search. Clearly, we then reduce the size of the searched state space to be exponential in k . It has been shown that many classical planning problems, e.g. problems from the International Planning Competition (IPC) domains, can be solved efficiently using width-based search with very low values of k .

The essential benefit of using width-based algorithms is the ability to perform semi-structured (based on feature structures) exploration of the state space, and reach deep states that may be important for achieving the planning goals. In classical planning, width-based search has been

integrated with heuristic search methods, leading to Best-First Width Search (?) that performed well at the 2017 IPC. Width-based search has also been adapted to reward-driven problems where the algorithm uses a simulator for interacting with the environment (?). This has enabled the use of width-based search in reinforcement learning environments such as the Atari 2600 video game suite, through the Arcade Learning Environment (ALE) (?). There have been several implementations of width-based search directly using the RAM states of the Atari computer as features (??).

Motivated by the fact that humans do not have access to RAM states when playing video games, methods for using these algorithms from screen pixels have been developed. ? propose a modified version of IW, called RolloutIW, that uses pixel-based features and achieves results comparable to learning methods in almost real-time on ALE. The combination of reinforcement learning and RolloutIW in the pixel domain is explored by ?, who propose to train a policy to guide the action selection in rollouts.

Width-based algorithms are highly exploratory and, not surprisingly, significantly dependent on the quality of the features defined. ? use a set of low-level, color-based features called B-PROST (?), designed by humans to achieve good performance in ALE. Integrating feature learning into the algorithms themselves rather than using hand-crafted features is an interesting challenge—and a challenge that will bring the algorithms more on a par with humans learning to play those video games. The features used by ? were the output features of the last layer of the policy network, improving the performance of IW. With the attempt to learn efficient features for width-based search, and inspired by the results of ?, this paper investigates the possibility of a more structured approach to generate features for width-based search using deep generative models.

More specifically, in this paper we use variational autoencoders (VAE)—latent variable models that have been widely used for representation learning (?) as they allow for approximate inference of the latent variables underlying the data—to learn propositional symbols directly from pixels and without any supervision. We then investigate whether the learned symbolic representation can be successfully used for planning in the Atari 2600 domain.

We compare the planning performance of RolloutIW using our learned representations with RolloutIW using the

*Equal contribution.

handcrafted B-PROST features and report human scores as a baseline. Our results show that our learned representations lead to better performance than B-PROST on RolloutIW (and often outperform human players), although the B-PROST features also contain temporal information (how the screen changes), whereas we only extract static features from individual frames. Apart from improving scores in the Atari 2600 domain, we also significantly reduce the number of features (by a factor of more than 10^3). We also investigate in detail (with ablation studies) which factors seem to lead to good performance on games in the Atari domain.

The main contributions of our paper are hence:

- We propose a novel combination of variational autoencoders and width-based planning algorithms.
- We train VAEs to learn propositional symbols for planning, from pixels and without any supervision.
- We run a large-scale evaluation on Atari 2600 where we compare the performance of RolloutIW with our learned representations to RolloutIW with B-PROST features.
- We investigate with ablation studies the effect of various hyperparameters in both learning and planning.
- We show that our learned representations lead to higher scores than using B-PROST, even though (i) our features don't have any temporal information and (ii) our models are trained from data collected by RolloutIW with B-PROST, thus limiting the richness of the dataset.

Below, we provide the required background on RolloutIW, B-PROST, and VAEs, present the original approach of this paper, discuss our experimental results, and conclude.

Background

Iterated width. Iterated Width (IW) (?) is a blind-search planning algorithm in which the states are represented by sets of boolean features (sets of propositional atoms). The set of all features/atoms is the *feature set*, denoted F . IW is an algorithm parameterized by a *width parameter* k . We use $IW(k)$ to denote IW with width parameter k . Given an initial state s_0 , $IW(k)$ is similar to a standard breadth-first search (BFS) from s_0 except that, when a new state s is generated, it is immediately pruned if it is not novel. A state s generated during search is defined to be *novel* if there is a k -tuple of features (atoms) $t = (f_1, \dots, f_k) \in F^k$ such that s is the first generated state making all of the f_i true (s making f_i true of course simply means $f_i \in s$, and the intuition is that s then “has” the feature f_i). In particular, in $IW(1)$, the only states that are not pruned are those that contain a feature $f \in F$ for the first time during the search. The maximum number of generated states is exponential in the width parameter ($IW(k)$ generates $O(|F|^k)$ states), whereas in BFS it is exponential in the number of features/atoms.

Rollout IW. RolloutIW(k) (?) is a variant of $IW(k)$ that searches via rollouts instead of doing a breadth-first search, hence making it more akin to a depth-first search. A *rollout* is a state-action trajectory $(s_0, a_0, s_1, a_1, \dots)$ from the initial state s_0 (a path in the state space), where actions a_i are

picked at random. It continues until reaching a state that is not novel. A state s is *novel* if it satisfies one of the following: 1) it is distinct from all previously generated states and there exists a k -tuple of features $(f_1, \dots, f_k)$ that are true in s , but not in any other state of the same depth of the search tree or lower; or 2) it is an already earlier generated state and there exists a k -tuple of features $(f_1, \dots, f_k)$ that are true in s , but not in any other state of lower depth in the search tree. The intuition behind case 1 is that the new state s comes with a combination of k features occurring at a lower depth than earlier encountered, which makes it relevant to explore further. In case 2, s is an existing state containing the lowest occurrence of one of the combinations of k features, again making it relevant to explore further. When a rollout reaches a state that is not novel, the rollout is terminated, and a new rollout is started from the initial state. The process continues until all possible states are explored, or a given time limit is reached. After the time limit has been reached, an action with maximal expected reward is chosen (the method was developed in the context of the Atari 2600 domain, making use of a simulator to compute action outcomes and rewards). The chosen action is executed, and the algorithm is repeated with the new state as the initial state.

RolloutIW is an anytime algorithm that will return an action independently of the time budget. In principle, IW could also be used as an anytime algorithm in the context of the Atari 2600 domain, but it will be severely limited by the breadth-first search strategy that will in practice prevent it from reaching nodes of sufficient depth in the state space, and hence prevent it from discovering rewards that only occur later in the game (?).

Planning with pixels. The Arcade Learning Environment (ALE) (?) provides an interface to Atari 2600 video games, and has been widely used in recent years as a benchmark for reinforcement learning and planning algorithms. In the visual setting, the sensory input consists of a pixel array of size 210×160 , where each pixel can have 128 distinct colors. Although in principle the set of $210 \times 160 \times 128$ booleans could be used as features, we will follow ? and focus on features that capture meaningful structures from the image.

An example of a visual feature set that has proven successful is *B-PROST* (?), which consists of *Basic*, *B-PROS*, and *B-PROT* features. The screen is split into 14×16 disjoint tiles of size 15×10 . For each tile (i, j) and color c , the basic feature $f_{i,j,c}$ is 1 iff c is present in (i, j) . A B-PROS feature $f_{i,j,c,c'}$ is 1 iff color c is present in tile t , color c' in tile t' , and the relative offsets between the tiles are i and j . Similarly, a B-PROT feature $f_{i,j,c,c'}$ is 1 iff color c is present in tile t in the previous decision point, color c' is present in tile t' in the current one, and the relative offsets between the tiles are i and j . The number of features in B-PROST is the sum of the number of features in these 3 sets, in total 20,598,848.

Variational autoencoders. *Latent variable models* (LVMs) are probabilistic models with unobserved variables. The marginal distribution over one observed datapoint $\mathbf{x}$ is $p_\theta(\mathbf{x}) = \int_{\mathbf{z}} p_\theta(\mathbf{x}, \mathbf{z}) d\mathbf{z}$, with $\mathbf{z}$ the unobserved *latent vari-*

ables and θ the model parameters. This quantity is typically referred to as *marginal likelihood* or *model evidence*. A simple and rather common structure for LVMs is:

$$p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x} | \mathbf{z}) p_\theta(\mathbf{z}).$$

If the model’s distributions are parameterized by neural networks, the marginal likelihood is typically intractable for lack of an analytical solution or a practical estimator.

Variational inference (VI) is a common approach to approximating this intractable posterior, where we define a distribution $q_\phi(\mathbf{z} | \mathbf{x})$ with *variational parameters* ϕ . In the LVM above, for any choice of q_ϕ we have:

$$\begin{aligned} \log p_\theta(\mathbf{x}) &= \log \mathbb{E}_{q_\phi(\mathbf{z} | \mathbf{x})} \left[\frac{p_\theta(\mathbf{x} | \mathbf{z}) p_\theta(\mathbf{z})}{q_\phi(\mathbf{z} | \mathbf{x})} \right] \\ &\geq \mathbb{E}_{q_\phi(\mathbf{z} | \mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x} | \mathbf{z}) p_\theta(\mathbf{z})}{q_\phi(\mathbf{z} | \mathbf{x})} \right] = \mathcal{L}_{\theta, \phi}(\mathbf{x}) \end{aligned}$$

where $\mathcal{L}_{\theta, \phi}(\mathbf{x})$ is a lower bound on the marginal log likelihood also known as *Evidence Lower BOund* (ELBO).

In contrast to traditional VI methods, where per-datapoint variational parameters are optimized separately, *amortized variational inference* utilizes function approximators like neural networks to share variational parameters across datapoints and improve learning efficiency. In this setting, $q_\phi(\mathbf{z} | \mathbf{x})$ is typically called an *inference model* or *encoder*. Variational Autoencoders (VAEs) (??) are a framework for amortized VI, in which the ELBO is maximized by jointly optimizing the inference model and the LVM (i.e., ϕ and θ , respectively) with stochastic gradient ascent.

VAE optimization. The ELBO, the objective function to be maximized, can be decomposed as follows:

$$\begin{aligned} \mathcal{L}_{\theta, \phi}(\mathbf{x}) &= \mathbb{E}_{q_\phi} [\log p_\theta(\mathbf{x} | \mathbf{z})] - \mathbb{E}_{q_\phi} \left[\log \frac{q_\phi(\mathbf{z} | \mathbf{x})}{p_\theta(\mathbf{z})} \right] \quad (1) \\ &= \mathbb{E}_{q_\phi} [\log p_\theta(\mathbf{x} | \mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) || p_\theta(\mathbf{z})) \end{aligned}$$

where the first term can be interpreted as negative expected *reconstruction error*, and the second term is the KL divergence from the prior $p_\theta(\mathbf{z})$ to the approximate posterior.

When optimizing VAEs, the gradients of some terms of the objective function cannot be backpropagated through the sampling step. However, for a rather wide class of probability distributions (?), a random variable following such a distribution can be expressed as a differentiable, deterministic transformation of an auxiliary variable with independent marginal distribution. For example, if z is a sample from a Gaussian random variable with mean $\mu_\phi(x)$ and standard deviation $\sigma_\phi(x)$, then $z = \sigma_\phi(x) \epsilon + \mu_\phi(x)$, where $\epsilon \sim \mathcal{N}(0, 1)$. Thanks to this *reparameterization*, z can be differentiated with respect to ϕ by standard backpropagation. This widely used approach, called *pathwise gradient estimator*, typically exhibits lower variance than the alternatives.

From an information theory perspective, optimizing the variational lower bound (1) involves a tradeoff between rate and distortion (?). A straightforward way to control the rate–distortion tradeoff is to use the β -VAE framework (?), in

which the training objective (1) is modified by scaling the KL term:

$$\begin{aligned} \mathcal{L}_{\theta, \phi, \beta}(\mathbf{x}) &= \mathbb{E}_{q_\phi} [\log p_\theta(\mathbf{x} | \mathbf{z})] \\ &\quad - \beta D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) || p_\theta(\mathbf{z})). \end{aligned} \quad (2)$$

VAEs with discrete variables. Since the categorical distribution is not reparameterizable, training VAEs with categorical latent variables is generally impractical. A solution to this problem is to replace samples from a categorical distribution with samples from a Gumbel-Softmax distribution (??), which can be smoothly annealed into a categorical distribution by making the temperature parameter τ tend to 0. Because Gumbel-Softmax is reparameterizable, the pathwise gradient estimator can be used to get a low-variance—although in this case biased—estimate of the gradient. In this work, we use Bernoulli latent variables (categorical with 2 classes) and their Gumbel-Softmax relaxations.

VAE-IW

Although width-based planning has been shown to be generally very effective for classical planning domains (???), its performance in practice significantly depends on the quality of the features used. Features that better capture meaningful structure in the environment typically translate to better planning results (?). Here we propose VAE-IW, in which representations extracted by a VAE are used as features for RolloutIW. The main advantages are:

- Significantly fewer features than B-PROST, leading to faster planning and a smaller memory footprint.
- Autoencoders, in particular VAEs, are a natural approach for learning meaningful, compact representations from data (???).
- No additional preprocessing is needed, such as background masking (?).

On the planning side, we follow ? and use RolloutIW(k) with width $k = 1$ to keep planning efficient even with a small time budget. In the remainder of this section, we detail the two main components of the proposed method: feature learning and planning.

Unsupervised feature learning. For feature learning we use a variational autoencoder with a discrete latent space:

$$p_\theta(\mathbf{x}) = \sum_{\mathbf{z}} p_\theta(\mathbf{x}, \mathbf{z}) = \sum_{\mathbf{z}} p_\theta(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) \quad (3)$$

where the prior is a product of independent Bernoulli distributions with a fixed parameter: $p(\mathbf{z}) = \prod_{i=1}^K p(z_i)$, where $p(z_i) = \text{Bernoulli}(\mu)$. Given an image $\mathbf{x}$, we would like to retrieve the binary latent factors $\mathbf{z}$ that generated it, and use them as a propositional representation for planning. Since $p_\theta(\mathbf{x} | \mathbf{z})$ is parameterized by a neural network, and therefore highly nonlinear with respect to the latent variables, inference of the latent variables is intractable (it would require the computation of the sum in (3), which has a number of terms exponential in the number of latent variables).

We define an inference model:

$$q_\phi(\mathbf{z} | \mathbf{x}) = \prod_{i=1}^K q_\phi(z_i | \mathbf{x}) = \prod_{i=1}^K \text{Bernoulli}((\mu_\phi(\mathbf{x}))_i)$$

to approximate the true posterior $p_\theta(\mathbf{z} | \mathbf{x})$. The *encoder* μ_ϕ is a deep neural network with parameters ϕ that outputs the approximate posterior probabilities of all latent variables given an image $\mathbf{x}$. Using stochastic amortized variational inference, we train the inference and generative models end-to-end by maximizing the ELBO with respect to the parameters of both models. We approximate the discrete Bernoulli distribution of $\mathbf{z}$ with the Gumbel-Softmax relaxation (??), which is reparameterizable. This allows us to estimate the gradients of the inference network with the pathwise gradient estimator. Alternatively, the approximate posterior could be optimized directly using a score-function estimator (?) with appropriate variance reduction techniques or alternative variational objectives (?????).

Note that we are not interested in the best generative model—e.g. in terms of marginal log likelihood of the data or visual quality of the generated samples—as much as in representations that are useful for planning. Thus, ideally, we would directly optimize β for planning performance. In practice, however, we assume that a necessary (but not sufficient) condition for a representation to be useful in our case is that it contains enough information about the entities on the screen. Following ?, we control the tradeoff between the competing terms in (1) by using (2) and decreasing β until good quality reconstructions are obtained.

Planning with learned features. After training, the inference model $q_\phi(\mathbf{z} | \mathbf{x})$ yields approximate posterior distributions of the latent variables. The binary features for downstream planning can be obtained by sampling from these distributions or by deterministically thresholding their probabilities given a fixed threshold $\lambda \in (0, 1)$. In this work we choose the latter approach as it empirically yields more stable features for planning. Overall, this provides a way of efficiently computing a compact, binary representation of an image, which can in turn be interpreted as a set of propositional features to be used in symbolic reasoning systems like IW or RolloutIW.

The planning phase in VAE-IW is based on RolloutIW, but uses the features extracted by the inference network instead of the B-PROST features. Note that while the B-PROST features include temporal information, the VAE features are computed independently for each frame. This should, everything else being equal, give an advantage to planners that rely on B-PROST features.

Experiments

We evaluated the proposed approach on a suite of 55 Atari 2600 games.¹ We separately trained a VAE for each domain by maximizing Eq. (2) using stochastic gradient ascent. To train the VAEs, we collected 15,000 frames by running RolloutIW(1) with B-PROST features, and split them into a training and validation set of 14,250 and 750 images.

¹Code available at <https://github.com/fred5577/VAE-IW>

The encoder consists of 3 convolutional layers interleaved with residual blocks. The single convolutional layers down-sample the feature map by a factor of 2, and each residual block includes 2 convolutional layers, resulting in 7 convolutional layers in total. The decoder mirrors such architecture, and uses transposed convolutions for upsampling. Further architectural details are provided in the Appendix. The inference network outputs a feature map of shape $H \times W \times C$ which represents the approximate posterior probabilities of the latent variables. Thus, unlike traditional VAEs where latent variables do not carry any spatial meaning, in our case they are spatially arranged (?).

As outlined in the previous section, the Bernoulli probabilities computed by the encoder are thresholded, and the resulting binary features are used as propositional representations for planning in RolloutIW(1), instead of the B-PROST features. For the planning phase of VAE-IW(1), we use RolloutIW(1) with partial caching and risk aversion (RA) as described by ?. With partial caching, after each iteration of RolloutIW(1), the branch of the chosen action is kept in memory. Risk aversion is attained by multiplying all negative rewards by a factor $\alpha \gg 1$ when they are propagated up the tree.

Because of their diversity, Atari games vary widely in complexity. We empirically chose a set of hyperparameters that performed reasonably well on a small subset of the games, assuming this would generalize well enough to other games. Following previous work (??), we used a frame skip of 15 and a planning budget of 0.5s per time step. We set an additional limit of 15,000 executed actions for each run, to prevent runs from lasting too long (note that this constraint is only applied to our method). We expect that better results can be obtained by performing an extensive hyperparameter search on the whole Atari suite. See the Appendix for further implementation details.

Main Results

In Table 1, we compare the planning performance of our RA VAE-IW(1) to IW(1) with B-PROST features, RA RolloutIW(1) with B-PROST features, and human play (?). In Figure 1, we normalize the scores of width-based planning methods using scores from random and human play, as in ?. Note that human play is only meant as a baseline: since humans do not have access to a simulator, a direct comparison cannot be made. Following the literature, we report the average score over 10 runs, with each run ending when the game is over. These results show that learning features purely from images, without any supervision, can be beneficial for width-based planning. In particular, using the learned representations in RolloutIW(1) leads to generally higher scores in the Atari 2600 domain. For reference, in ?? we provide an additional comparison of our method with other related approaches. However, these significantly differ from VAE-IW in some aspects and are therefore not directly comparable.

Note that the performance of VAE-IW depends on the quality and expressiveness of the features extracted by the VAE, which in turn depend on the images the VAE is trained on. Crucially, since we collect data by running RolloutIW

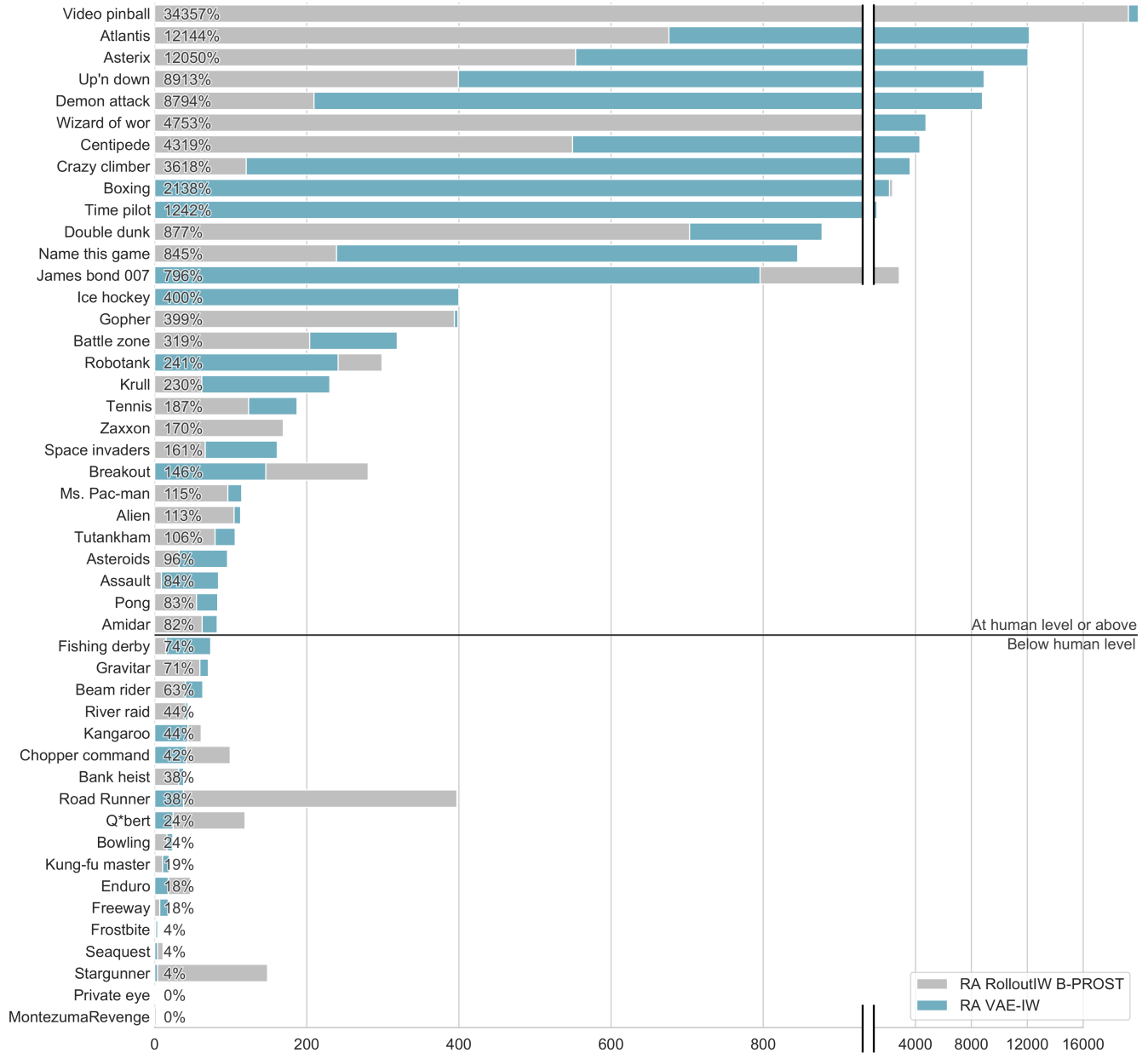


Figure 1: Comparison of the risk-averse variants of VAE-IW(1) (ours, using VAE features) and RolloutIW(1) B-PROST (using B-PROST features). Following VAE-IW , the performance of both methods is normalized with respect to a professional human game tester (100% level) and random play (0%) as: $100 \times (\text{VAE} - \text{random play}) / (\text{human score} - \text{random play})$. RA VAE-IW(1) obtains the highest score among width-based approaches in most games, and it performs at a level that is superior to or comparable with professional human play. The reported percentages are for VAE features.

with B-PROST features, the performance of VAE-IW is constrained by the effectiveness of the data collection algorithm. The VAE features will not necessarily be meaningful on parts of the game that are significantly different from those in the training set. Surprisingly, however, our method significantly outperforms the baseline that was used for data collection, providing even stronger evidence that the compact

set of features learned by a VAE can be successfully utilized in width-based planning. This form of bootstrapping can be iterated: images collected by VAE-IW could be used for re-training or fine-tuning VAEs, potentially leading to further performance improvements. Although in principle the first iteration of VAEs could be trained from images collected via random play, this is not a viable solution in hard-exploration

Algorithm Features	Human	IW B-PROST	RolloutIW B-PROST	VAE-IW VAE
Alien	6,875	1,316	7,170	7,744
Amidar	1,676	48	1,049.2	1,380.3
Assault	1,496	268.8	336	1,291.9
Asterix	8,503	1,350	46,100	999,500*
Asteroids	13,157	840	4,698	12,647
Atlantis	29,028	33,160	122,220	1,977,520*
BankHeist	734.4	24	242	289
BattleZone	37,800	6,800	74,600	115,400
BeamRider	5,775	715.2	2,552.8	3,792
Berzerk		280	1,208	863
Bowling	154.8	30.6	44.2	54.4
Boxing	4.3	99.4	99.2	89.9
Breakout	31.8	1.6	86.2	45.7
Centipede	11,963	88,890	56,328	428,451.5*
ChopperCommand	9,882	1,760	9,820	4,190
CrazyClimber	35,411	16,780	40,440	901,930*
DemonAttack	3,401	106	6,958	285,867.5*
DoubleDunk	-15.5	-22	3.2	8.6
ElevatorAction		1,080	0	40,000*
Enduro	309.6	2.6	145.8	55.5
FishingDerby	5.5	-83.8	-77	-20
Freeway	29.6	0.6	2	5.3
Frostbite	4,335	106	146	259
Gopher	2,321	1,036	8,388	8,484
Gravitar	2,672	380	1,660	1,940
IceHockey	0.9	-13.6	-12.4	37.2
Jamesbond	406.7	40	10,760	3,035
Kangaroo	3,035	160	1,880	1,360
Krull	2,395	3,206.8	2,091.8	3,433.9
KungFuMaster	22,736	440	2,620	4,550
MontezumaRevenge	4,367	0	100	0
MsPacman	15,693	2,578	15,115	17,929.8
NameThisGame	4,076	7,070	6,558	17,374
Phoenix		1,266	6,790	5,919
Pitfall		-8.6	-302.8	-5.6
Pong	9.3	-20.8	-4.2	4.2
PrivateEye	69,571	2,690.8	-480	80
Qbert	13,455	515	15,970	3,392.5
Riverraid	13,513	664	6,288	6,701
RoadRunner	7,845	200	31,140	2,980
Robotank	11.9	3.2	31.2	25.6
Seaquest	20,182	168	2,312	842
Skiing		-16,511	-16,006.8	-10,046.9
Solaris		1,356	1,704	7,838
SpaceInvaders	1,652	280	1,149	2,574
StarGunner	10,250	840	14,900	1,030
Tennis	-8.9	-23.4	-5.4	4.1
TimePilot	5,925	2,360	3,540	32,840
Tutankham	167.7	71.2	135.6	177
UpNDown	9,082	928	34,668	762,453*
Venture	1,188	0	60	0
VideoPinball	17,298	28,706.4	216,468.6	373,914.3
WizardOfWor	4,757	5,660	43,860	199,900
YarsRevenge		6,352.6	7,848.8	96,053.3
Zaxxon	9,173	0	15,500	15,560
# > Human	n/a	4	22	25
# > 75% Human	n/a	4	26	28
# best in game	n/a	2	14	39

Table 1: Score comparison in 55 Atari games. Scores in bold indicate the best width-based method, and a star indicates that the limit on executed actions was reached at least once. RolloutIW and VAE-IW are run with risk aversion.

problems such as many Atari games (?).

In addition, there appears to be a significant negative correlation between the average number of true features and

the average number of expanded nodes per planning phase (Spearman rank correlation of -0.51 , p -value < 0.001). In other words, in domains where the VAE extracts on average more true features, the planning algorithm tends to expand fewer nodes. Thus, it could be potentially fruitful to further investigate the interplay between the average number of true features, the meaningfulness and usefulness of the features, and the efficiency of the width-based planning algorithm.

Ablation Studies

The performance of VAE-IW depends on hyperparameters of the planning algorithm and the probabilistic model. Here we investigate the effect of some of these parameters. Tables with full results are reported in the Appendix.

Modeling choices. One of the major modeling choices is the dimensionality of the latent space, and the spatial structure $H \times W \times C$ of the latent variables. Both of these factors are tightly coupled with the neural architecture underlying the inference and generative networks. As there is no clear heuristic, we explored different neural architectures and latent space sizes. Based on the performance on a few selected domains, we chose two representative settings, with latent space size $15 \times 15 \times 20$ and $4 \times 4 \times 200$ (see the Appendix for further details). In Table ?? we compare the performance of RA VAE-IW(1) on these two configurations, keeping the rest of the hyperparameters fixed as the ones used in Table 1. While overall the $15 \times 15 \times 20$ configuration leads to a higher score in most domains, the effect of this modeling choice seems to significantly depend on the domain.

An alternative to directly training a discrete latent variable model is to train a continuous one and then quantize the inferred representations at planning time. We investigated this possibility by training a continuous VAE with latent space size $15 \times 15 \times 5$ and independent prior $\mathcal{N}(0, 1)$ for each latent variable. When planning, we quantize the inferred posterior means into 2^4 or 2^6 bins of equal probability under the prior. This leads to 4 or 6 bits per latent variable, and hence 4,500 and 6,750 boolean features, respectively. In Table ??, we observe that this approach, although generally competitive, tends to be inferior to training a discrete model directly.

As previously mentioned, we consider the framework of β -VAEs in which β controls the trade-off between the reconstruction accuracy and the amount of information encoded in the latent space. For our purposes, β has to be small enough that the model can capture all relevant details in an image. In practice, we decreased β until the model generated sufficiently accurate reconstructions on a selection of Atari games. Table ?? reports the performance of VAE-IW(1) when varying $\beta \in \{10^{-4}, 10^{-3}\}$, and shows that the effect of varying β depends on the domain. Intuitively, while a higher β can be detrimental for the reconstructions and thus also for the informativeness of the learned features, it may lead to better representations in less visually challenging domains. In practice, one could for example train VAEs with different regularization strengths, and do unsupervised model selection separately for each domain by using the “elbow method” (?) on the reconstruction loss or other relevant

metrics.

Planning parameters. Regardless of the features used for planning, the performance of VAE-IW depends on the variant of RolloutIW being used, and on its parameters. In Table ?? we compare the average score of VAE-IW(1) with and without risk aversion and observe that the risk-averse variant achieves an equal or higher average score in 68% of the domains. Table ?? shows the results of VAE-IW(1) and RolloutIW(1) with B-PROST features, similarly to Table 1, except that both methods are run *without* risk aversion. With this modification, our method still obtains a higher average score in the majority of games (68%).

Another crucial planning parameter is the time budget for planning at each time step. While the main results are based on a 0.5s budget, we also consider a 32s budget, following ?. In Table ?? we observe that, unsurprisingly, the 32s budget leads to better results in most domains (79%). Interestingly, increasing the planning budget does not seem to affect the average rollout depth, while the average number of expanded nodes in each planning phase grows significantly. This behaviour is consistently observed in all tested domains (see Figures ?? and ?? in Appendix) and points to the fact that increasing the planning budget improves results mostly by allowing more rollouts.

In Table ?? we compare the average scores obtained by VAE-IW(1) and RolloutIW(1) with B-PROST features, using a 32s planning budget. Once again, using the compact features learned by a VAE seems to be beneficial, as VAE-IW(1) obtains the highest average score in 62% of the games. However, the margin here is smaller than in the experiments with a budget of 0.5s, where VAE-IW(1) outperformed RolloutIW(1) B-PROST in 73% of the games.

Related Work

Variational Autoencoders (VAEs) have been extensively used for representation learning (?) as their amortized inference network lends itself naturally to this task. In the context of automated planning, ? proposed a State Autoencoder (SAE) for propositional symbolic grounding. SAEs are in fact VAEs with Bernoulli latent variables. They are trained by maximizing a modified ELBO with an additional entropy regularizer, defined as twice the negative KL divergence. Thus, the objective function being maximized is the ELBO (1) with the sign of the KL flipped. A variation of SAE, the zero-suppressed state autoencoder (?), adds a further regularization term to the propositional representation, leading to more stable features.

? learn state features with supervised learning, and then plan in the feature space with a learned transition graph. ? specify a set of skills for a task, and automatically extract state representations from raw observations. ? use generative adversarial networks to learn structured representations of images and a deterministic dynamics model, and plan with graph search. ? proposed π -IW, a variant of RolloutIW where a neural network guides the action selection process in the rollouts, which would otherwise be random. More-

over, π -IW plans using features obtained from the last hidden layer of the policy network, instead of B-PROST.

Atari games have also been extensively used as a testbed in model-free (?????) and model-based (????) deep reinforcement learning.

Conclusions

We have introduced a novel combination of width-based planning with learning techniques. The learning employs a VAE to learn relevant features in Atari games given images of screen states as training data. The planning is done with RolloutIW(1) using the features learned by the VAE. Our approach reduces the size of the feature set from the 20.5 million B-PROST features used in previous work in connection with RolloutIW, to only 4,500. Our algorithm, VAE-IW, outperforms the previous methods, proving that VAEs can learn meaningful representations that can be effectively used for planning with RolloutIW.

In VAE-IW, the symbolic representations are learned from data collected by RolloutIW using B-PROST features. Increasing the diversity and quality of the training data could potentially lead to better representations, and thus better planning results. One possible way to achieve this could be to iteratively retrain and fine-tune the VAEs based on data collected by the current iteration of VAE-IW. The quality of the representations could also be improved by using more expressive discrete models, for example with a hierarchy of discrete latent variables (?). Similarly, in the ablation study where we compare this approach with quantizing continuous representations, more flexible models (??) and more advanced quantization strategies could be employed. Finally, we can expect further improvements orthogonal to this work, by learning a rollout policy for more effective action selection as in ?.

Molestiae suscipit praesentium cum incidunt sapiente, odio tenetur id doloribus explicabo?Fuga enim maiores dolore, minima quidem odit, corrupti placeat at neque totam consequuntur modi voluptate atque ab repellendus, dignissimos facere vitae iste?Vel quaerat assumenda nemo sint ea voluptatem impedit nobis voluptatum quam eligendi, odit ad voluptatibus id perspiciatis a natus unde sequi saepe necessitatibus, accusamus ab quidem itaque culpa?Earum asperiores est amet placeat quod quia deleniti adipisci aliquam impedit, doloremque architecto quos nisi?Expedita voluptatem facilis, fugiat vero labore, dolore libero in, perferendis inventore deleniti provident ratione nobis dolore voluptatibus explicabo, amet facilis cumque obcaecati aperiam dicta repudiandae?Vero molestias fugiat explicabo id inventore est fuga quo vel nemo, ex tempora sint cum minus optio?Ad quis ab saepe at illum ipsam tempora enim voluptatum, doloremque enim laborum dolorem eveniet neque repudiandae accusantium itaque doloribus.Doloribus dolore labore tempora quis, placeat minus dolore sit, ut asperiores libero voluptate atque natus ducimus corrupti veritatis?Nam non quasi voluptatem placeat harum velit maxime possimus molestiae enim perferendis, minus hic iste culpa obcaecati in dignissimos quos nemo optio qui sint, ad voluptatum quae tenetur repellat a odit debitis rerum aspernatur beatae, officia praesentium dolorum deleniti exercitationem eaque

non. Porro nisi corrupti corporis quisquam recusandae fuga
fugiat quaerat, distinctio repudiandae fugit repellendus in-
ventore sint iure maiores quis